

Agenda

- Class 16 – October 30th (1:00 – 3:00)
- Starter Quiz
- Observability, Multi Cloud , Continuous Delivery Model Architecture
- Thought Paper Presentation
 - Consumer Persona's – Startups and Entrepreneurs (Team 7)
- Observability, Continuous Delivery Model, Multi Cloud Architecture (Cont'd)

Note:

- Mobile Phones – Do not bring to class or Completely Switch off
- If students have any specific topic they want to discuss, share the topic and we can try to schedule
- Please be seated by your teams

Starter Quiz

- **What is largest selling Candy Bar 50 years back**
 - **Hint: Even today it is the largest selling Candy Bar**

Snickers



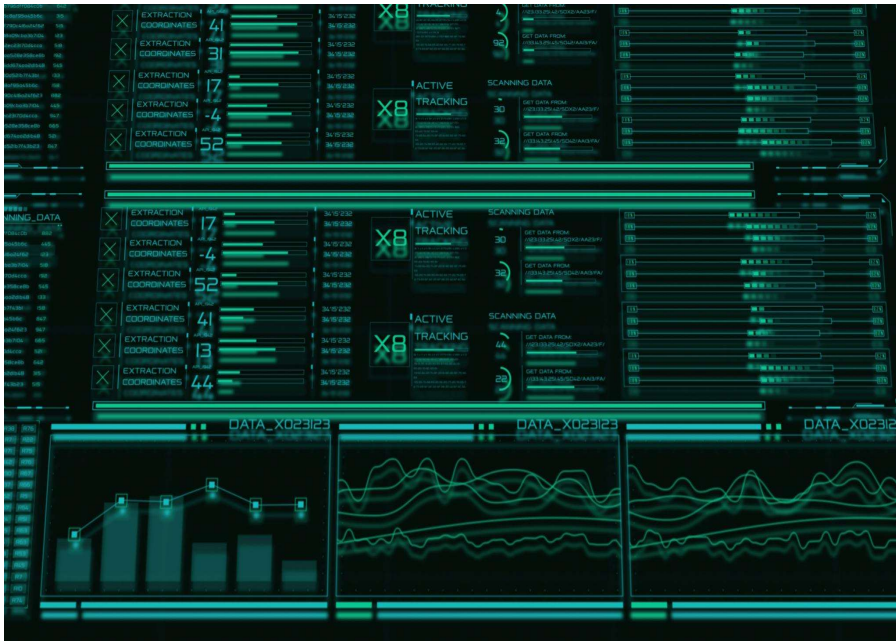
Core Banking Platform Engineering –
Observability, Continuous Delivery Model
and Multi Cloud Adaptable Architecture

Observability, Multi Cloud Adaptable and Continuous Delivery Model Architecture

- **Observability**
 - Overview
 - Open Telemetry
 - Observability Tools
- **Multi Cloud**
 - Overview
 - Key Principles
 - Challenges
- **Continuous Delivery Model**
 - Overview
 - Key Principles
 - Components
 - Rollout Recommendations
 - Success Tips

Observability

Observability Overview



- Real-Time System Monitoring
 - Observability architecture enables continuous monitoring by collecting real-time metrics, logs, and traces from distributed systems.
- Holistic System Visibility
 - Integrating diverse data sources provides a comprehensive view, allowing teams to understand the overall system health and performance.
- Enhanced Reliability and Experience
 - Robust observability supports faster issue resolution, improves reliability, and enhances the user experience in modern software environments.



Observability vs Monitoring

Understanding Observability

Observability gathers metrics, logs, and traces for deep system insights, supporting proactive detection and investigation of issues.

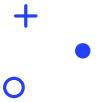
Role of Monitoring

Monitoring tracks health using thresholds, triggers alerts for anomalies, and helps teams react quickly to problems.

Integrating for Reliability

Combining observability and monitoring with integrated tools ensures robust system reliability and rapid troubleshooting.

OPEN TELEMETRY OVERVIEW



Standardized Telemetry Collection

Open Telemetry standardizes collection of metrics, logs, and traces from applications and infrastructure.

Vendor-Neutral Integration

Provides vendor-neutral APIs and SDKs enabling easy integration with various backend platforms.

Multi-Language Support

Supports many programming languages to facilitate broad adoption across diverse development environments.

Unified Observability Solution

Unifies telemetry tools to reduce complexity and avoid vendor lock-in, enhancing system insights.

OPEN TELEMETRY ARCHITECTURE

Core Components

Open Telemetry includes instrumentation libraries, a Collector, and exporters to manage telemetry data.

Instrumentation Methods

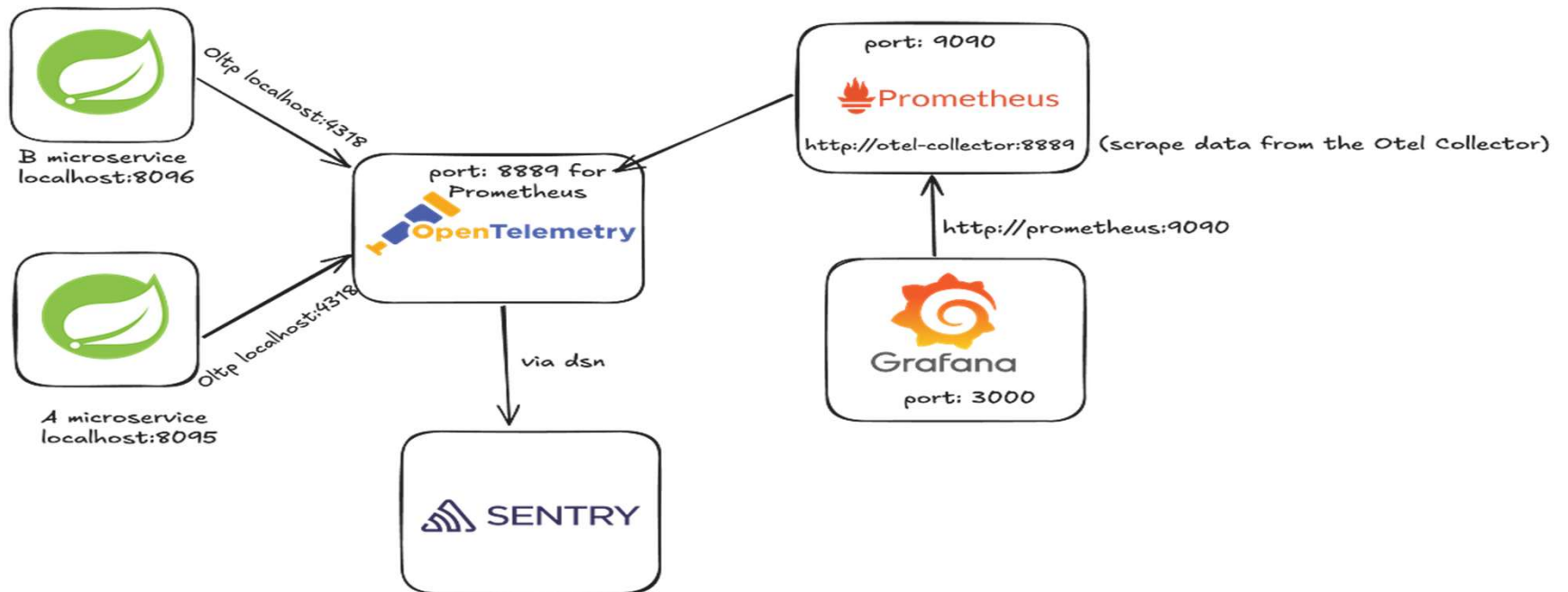
Applications use SDKs or auto-instrumentation agents to generate standardized telemetry data.

Collector Functions

The Collector receives, processes, and exports data using receivers, processors, and exporters.

Modular and Extensible Design

This design allows customization and scalability in observability pipelines for organizations.



KEY OBSERVABILITY TOOLS



Prometheus

Comprehensive Monitoring

Prometheus enables robust monitoring of applications and infrastructure by collecting vital metrics in real time from diverse software systems efficiently.

Integration with Open Telemetry

Supports native integration with Open Telemetry for standardized metrics scraping from applications

Time Series Data Collection

Metrics are stored as time series data, which supports efficient querying and analysis of system performance trends.

Advanced Alerting System

Enables sophisticated alerting using Alert manager to proactively notify teams about issues.

Opensource based

Prometheus is an open-source monitoring and alerting toolkit. It was originally developed at SoundCloud and later became a free-to-use open-source project managed by the Cloud Native Computing Foundation (CNCF)



Grafana

Time-Series Data Visualization

Grafana excels at displaying time-series data, making trends and patterns easy to analyze over time.

Multiple Data Sources Supported

It integrates with various data sources such as Prometheus and Elasticsearch for flexible data collection.

Opensource based

Grafana is an open-source data visualization and monitoring tool. The open-source version of Grafana is free to use, and it is licensed under the AGPL-3.0-only license.



Dynatrace: Intelligent Monitoring



Advanced Application Monitoring

Dynatrace enables comprehensive monitoring of application performance, helping teams quickly identify and resolve issues.

AI-Driven Issue Detection

AI technology in Dynatrace automatically detects and diagnoses performance issues, reducing downtime and manual intervention.

Full Stack Visibility and Enterprise Grade

Dynatrace provides end-to-end visibility across the entire technology stack, supporting IT operations and DevOps teams. Designed for large enterprises requiring advanced operational intelligence and observability.

Commercial Software and Expensive

Dynatrace is a proprietary, commercial software platform

Multi Cloud Architecture

Designing for Multi-Cloud Deployment

Avoid Vendor Lock in

Running on multiple clouds improves flexibility and resilience, helping organizations avoid dependence on a single vendor (i.e. Vendor Lock in).

Geography Considerations

Containerization and standards adherence (e.g. ANSI SQL Standards to enable running across Multiple Relational Databases across Cloud Providers) allow teams to move workloads easily between cloud environments.

Key Deployment Models

- Public Cloud – AWS, Azure, GCP
- On Prem – Banks Data Centre
- Hybrid Cloud – Development& Testing in Public Cloud, Production Operations On Prem



KEY PRINCIPLES OF MULTI-CLOUD APPLICATION DESIGN

Cloud-Agnostic Architecture

Use open standards to ensure portability across multiple cloud providers.

E.g. open protocols like REST, gRPC, and ANSI SQL to ensure multi-cloud compatibility.

Containerization for Consistency

Leverage Docker and Kubernetes to enable consistent deployments across clouds.

Cloud Services Abstraction

Abstracting the Cloud Services (e.g. Event Broker Service, Identity Management Service, Database Service ..) to application enables Integrating with the Specific Cloud Services of each Provider with minimal changes and with out any impact to Application Business Logic

KEY PRINCIPLES OF MULTI-CLOUD APPLICATION DESIGN

Automated CI/CD Pipelines

Use automated deployment pipelines for automated testing and efficient application delivery.

Infrastructure as Code

Use Terraform to provision infrastructure across multiple clouds from a single codebase.

OpenStack provides a single interface for managing infrastructure across clouds.

Use Open Source based Observability Tools

Implement monitoring and logging using Prometheus and Grafana that can be deployed on any Cloud

Designing for Multi-Cloud - Challenges

Application Performance Consistency

Services Nuances & variations among providers make it difficult to maintain consistent application performance in multi-cloud setups.

Security and Compliance

Maintaining security and compliance across different platforms is challenging, requiring unified yet adaptable policy management.

Provisioning, Specific Tooling, Certification, Release Costs

Provisioning Infrastructure, Cloud Specific Tooling and Certifying the application/platform across multiple clouds and Releasing the same could add up to significant costs and Complexity

Cloud Providers Varying Release and Support Cycles

Each Cloud Provider Release and Support Cycles for their Services Vary. Tracking and Keeping up pace with all these changes in parallel adds to the complexity application Development and Maintenance.



Continuous Delivery Model

CONTINUOUS DELIVERY MODEL OVERVIEW

Automated Release Process

Continuous Delivery automates the full release process from code commit to deployment, ensuring rapid and safe releases.

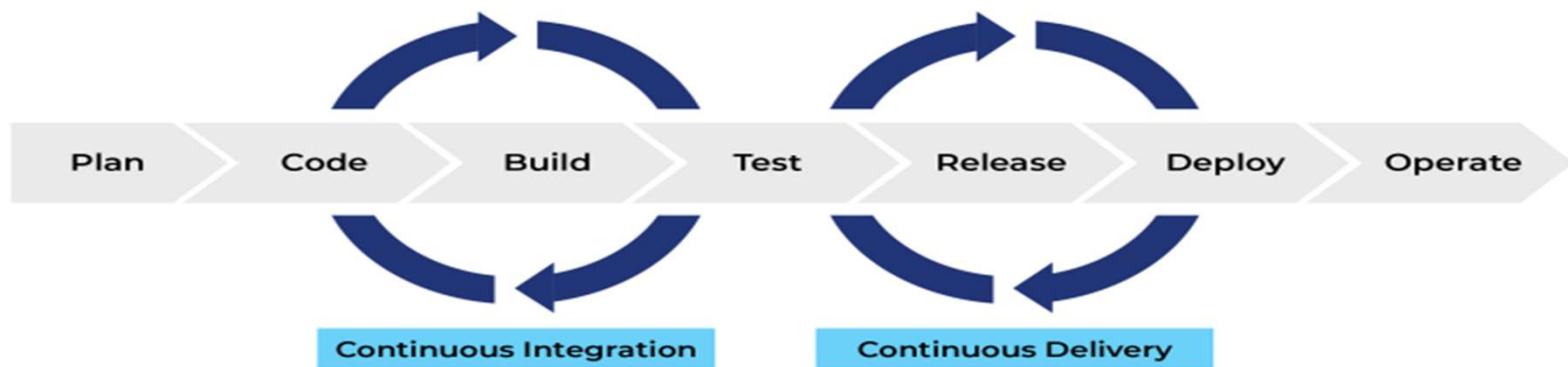
Rapid Iterations and Updates

CD supports frequent software updates and rapid iterations to meet dynamic user and market demands.

Reliability and Agility

By integrating automated testing, CD validates each code change, promoting reliable and agile software delivery.

CI/CD



CONTINUOUS DELIVERY MODEL OVERVIEW

Always Shippable Codebase

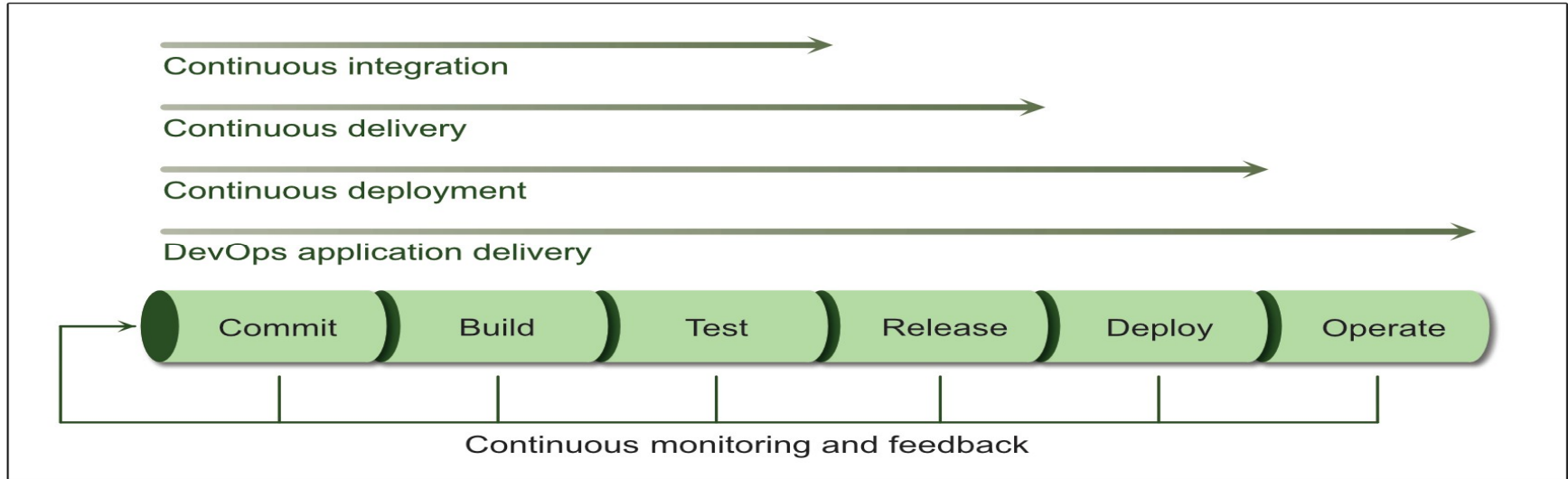
Maintain a main code branch ready for production deployment at any time to ensure continuous delivery.

Automated Testing

Implement unit, integration, and end-to-end tests to validate every code change automatically and reliably.

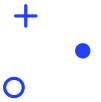
Deployment and Release Separation

Separate deployment from release using feature toggles to safely control feature activation and reduce risks.



CONTINUOUS DELIVERY MODEL

KEY PRINCIPLES



Source Control Management

Manages code versions ensuring organized and trackable software development.

Continuous Integration and Testing

Automates build and testing to validate software functionality and performance.

Continuous Deployment

Deploys software changes to staging and production environments seamlessly.

Monitoring and Feedback

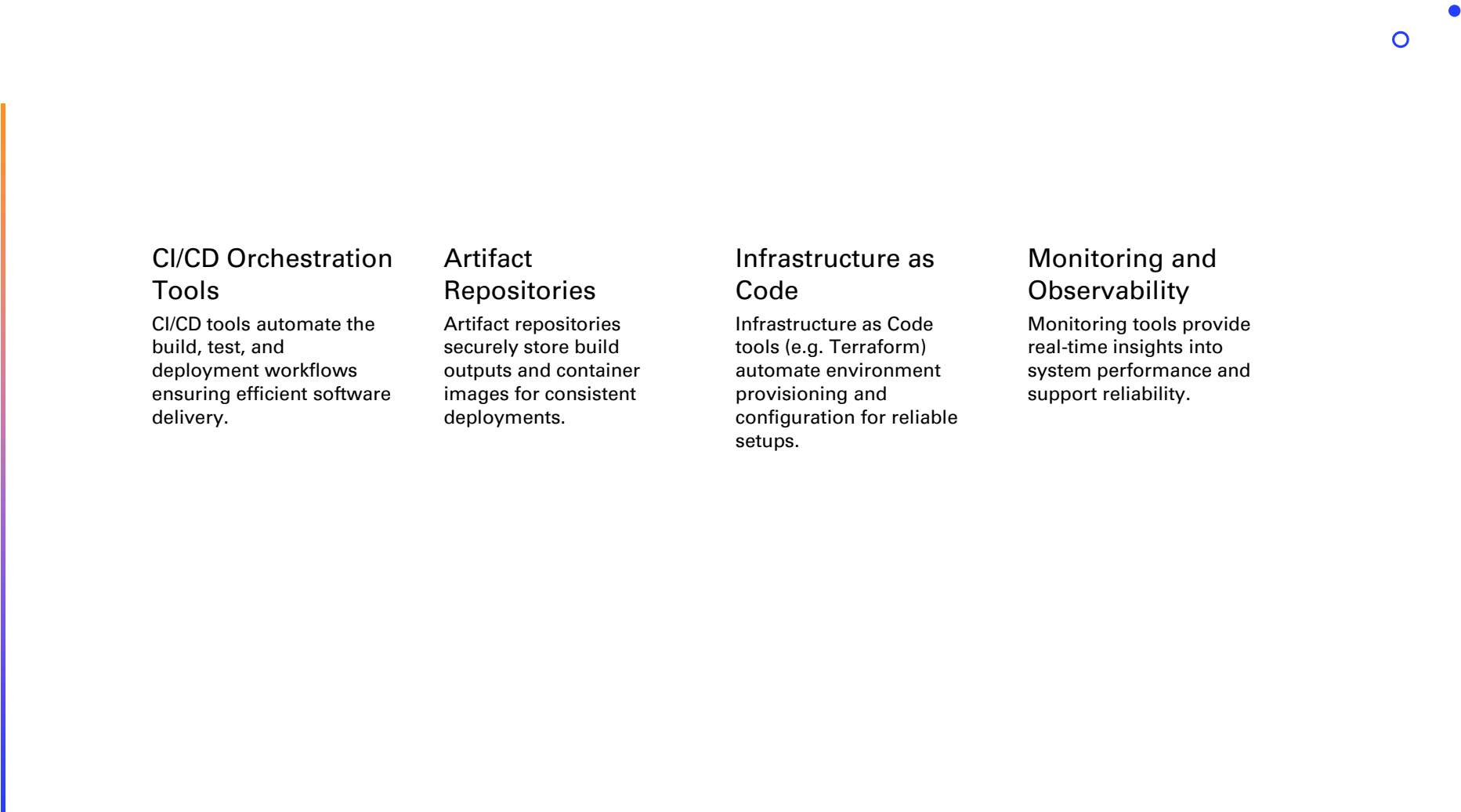
Provides real-time system health and user experience feedback to teams.

Continuous Delivery: the eight principles



Source: First Consulting

CONTINUOUS DELIVERY MODEL COMPONENTS

A decorative graphic on the left side of the slide consists of a vertical line that is orange at the top and transitions to blue at the bottom. To the right of this line, there is a small cluster of blue dots and a blue circle, arranged in a pattern that suggests a network or a flow.

CI/CD Orchestration Tools

CI/CD tools automate the build, test, and deployment workflows ensuring efficient software delivery.

Artifact Repositories

Artifact repositories securely store build outputs and container images for consistent deployments.

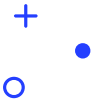
Infrastructure as Code

Infrastructure as Code tools (e.g. Terraform) automate environment provisioning and configuration for reliable setups.

Monitoring and Observability

Monitoring tools provide real-time insights into system performance and support reliability.

CONTINUOUS DELIVERY MODEL- ROLLOUT RECOMMENDATIONS



Enablement Phase

Establish infrastructure, governance, and prepare teams for the Continuous Delivery model adoption.

Implementation Phase

Deploy delivery model, integrate systems, and validate processes through rigorous testing.

Production & Operations Support

Monitor live operations, manage incidents, and optimize system performance continuously.

Release & Upgrade Phase

Deliver continuous improvements and feature enhancements ensuring ongoing product evolution.

Continuous Delivery Model

Success Tips

Key Practices

Use feature flags for safe releases, automate rollbacks, and monitor latency and error rates continuously.

Automate Pipelines

Automate testing and deployment pipelines for rapid, reliable software releases and improved delivery efficiency.

Strong Version Control

Maintain a robust version control system with consistent integration to minimize merge conflicts and streamline releases.

Collaboration Culture

Encourage collaboration between development and operations teams for faster feedback and continuous improvement.

